Tecnología CSS

¿Qué es CSS?

Definición

- **CSS (Cascading Style Sheets)** es el lenguaje que describe **cómo se ve** un documento HTML: colores, tipografía, espaciados, layout y adaptabilidad.

Separación de responsabilidades (idea conceptual)

- **HTML**: estructura y significado (semántica).
- **CSS**: presentación (estilo y disposición).
- **JS**: comportamiento (interacción, datos, lógica).

Cómo se "conecta" CSS con HTML

Tres formas habituales

1. **Archivo externo** (recomendado)

```
<link rel="stylesheet" href="styles.css">
```

2. **Bloque interno**

```
<style>
  body { font-family: system-ui; }
</style>
```

3. **Estilo en línea** (solo casos puntuales)

```
<p style="color:red">Hola</p>
```

Idea clave

- El CSS externo facilita **mantenimiento, cacheo y escalabilidad**.

Anatomía de una regla CSS

Estructura

```
selector {
  propiedad: valor;
  propiedad: valor;
}
```

Ejemplo

```
h1 {
  color: #222;
  font-size: 2rem;
}
```

Terminología

- `h1` → **selector**
- `{ ... }` → **bloque de declaraciones**
- `color` / `font-size` → **propiedades**
- `#222` / `2rem` → **valores**

Selectores básicos

Por etiqueta

```
p { line-height: 1.5; }
```

Por clase (muy común)

```
.card { padding: 1rem; }
```

Por id (evitar para estilos en general)

```
#hero { background: black; }
```

Cómo funcionan las clases en HTML

HTML

```
<article class="card featured">
  <h2 class="card-title">Título</h2>
</article>
```

CSS

```
.card { border: 1px solid #ddd; }
.featured { border-color: gold; }
.card-title { margin: 0; }
```

Idea clave

- Un elemento puede tener **varias clases** (se suman).

Selectores combinados (descendencia)

Caso típico: “estiliza esto solo dentro de aquello”

```
.card h2 { font-size: 1.25rem; }
```

Qué significa

- Aplica a `h2` **que estén dentro** de `.card`.

Advertencia (buena práctica inicial)

- No abuses de selectores muy largos: tienden a ser frágiles.

Pseudo-clases: estados e interacción

Hover / Focus

```
a:hover { text-decoration: underline; }
button:focus { outline: 2px solid; }
```

Formularios

```
input:invalid { border-color: red; }
input:disabled { opacity: 0.6; }
```

Idea clave

- Las pseudo-clases expresan **estados** (sin JavaScript).

Pseudo-elementos: “partes” virtuales

Antes y después

```
.badge::before { content: "★ "; }
.badge::after  { content: " ✓"; }
```

Uso típico

- Decoración, iconos simples, detalles visuales.

Regla importante

- Sin `content`, `::before` / `::after` no aparece.

El concepto más importante: la Cascada (Cascade)

Qué es

- CSS decide el estilo final combinando reglas según:
    1. **origen** (autor, navegador, usuario)
    2. **importancia** (`!important`)
    3. **especificidad**
    4. **orden** (lo último suele ganar)

La cascada: cómo CSS decide el estilo final (visión general)

Definición técnica

- La **cascada (cascade)** es el algoritmo con el que el navegador combina **todas** las reglas CSS aplicables a un elemento para obtener su estilo final.

Idea clave

- Cuando dos reglas compiten por la misma propiedad (ej. `color`), CSS resuelve el conflicto aplicando una jerarquía:

1. **origen** → 2) **importancia** → 3) **especificidad** → 4) **orden**

"¿Esta regla aplica?" (selector + condiciones)

Antes de la cascada, CSS filtra:

- **El selector coincide** con el elemento.
- Se cumple el contexto: media queries (`@media`), soportes (`@supports`), container queries (`@container`), etc.

Ejemplo

```
@media (min-width: 800px) {
  .btn { padding: 1rem; }
}
```

- Si la pantalla mide menos de 800px, esa regla ni siquiera entra en la competición.

Origen (autor, navegador, usuario)

Origen: quién escribió esa regla (prioridades)

Las reglas no vienen todas del mismo sitio:

1. **User Agent (navegador)**
    - Estilos por defecto: márgenes en `body`, tamaños de `h1`, subrayado en `a`, etc.
2. **Autor (tu CSS)**

    - Tus archivos `styles.css`, `<style>...</style>`, frameworks, etc.
3. **Usuario**
    - Ajustes del usuario: estilos del sistema o del navegador (por accesibilidad), extensiones, etc.

Regla práctica inicial

- Normalmente, tus estilos (autor) sobreescriben los del navegador.

Ejemplo (visible en cualquier web)

- El navegador subraya los enlaces por defecto.
- Si tú pones `a { text-decoration: none; }`, gana tu regla.

Por qué existe "origen"

Motivo técnico

- Permite que el navegador tenga un estilo mínimo funcional (sin CSS, la página se lee).
- Permite que el usuario pueda imponer preferencias críticas (accesibilidad).

Ejemplo conceptual

- Usuario con necesidades de contraste puede forzar configuraciones que prevalezcan sobre ciertos estilos.

Importancia (`!important`)

Importancia: `!important` cambia la jerarquía

Qué hace `!important`

- Marca una declaración como **más prioritaria** frente a declaraciones normales del mismo origen (y, en general, sube en la jerarquía de decisión).

Ejemplo

```
.title { color: blue; }
.title { color: green !important; }
```

- Gana **green**, incluso si otra regla aparece después (salvo que también sea `!important` y gane por especificidad/orden).

Cuando compiten dos `!important`

Si ambas reglas tienen `!important`, entonces CSS vuelve a aplicar:

- **especificidad → orden**

Ejemplo

```
.title { color: green !important; }      /* (0,1,0) */
#hero .title { color: red !important; } /* (1,1,0) */
```

- Gana **red** por mayor especificidad.

Por qué `!important` suele ser mala idea (ingeniería)

Efecto "escalada"

- Si alguien usa `!important`, otra persona lo "derrota" con otro `!important` más específico… y el sistema se vuelve inestable.

Uso legítimo (casos puntuales)

- Utilidades muy controladas (p.ej. `.u-hidden { display:none !important; }`)
- Overrides de librerías cuando no hay alternativa (idealmente con `@layer`, no con `!important`)

Especificidad: cuánto "pesa" un selector

Definición

- Es un sistema de puntuación para decidir qué selector es "más fuerte" cuando varias reglas compiten.

Modelo (a,b,c)

- **a**: IDs (`#id`)
- **b**: clases (`.x`), atributos (`[type="..."]`), pseudo-clases (`:hover`, `:focus`, `:has(...)`)
- **c**: etiquetas (`p`, `h1`), pseudo-elementos (`::before`)

Ejemplos

- `p` → (0,0,1)
- `.text` → (0,1,0)
- `#x` → (1,0,0)
- `#hero .text p` → (1,1,1)

Ejemplo completo de competición por especificidad

CSS

```
p { color: blue; }          /* (0,0,1) */
.text { color: green; }     /* (0,1,0) */
#x { color: red; }          /* (1,0,0) */
```

HTML

```
<p id="x" class="text">Hola</p>
```

Resultado

- Gana `#x` → **red**.

Errores frecuentes con especificidad

Error 1: "lo pongo más abajo y ya"

- No funciona si la regla de arriba tiene más especificidad.

Error 2: encadenar selectores para "ganar"

```
.page .content .card .title { ... } /* frágil y difícil de romper */
```

Solución típica

- Arquitectura: clases claras, capas (`@layer`), evitar IDs para estilo.

Orden entre archivos y entre importaciones

Caso típico

- Si cargas dos CSS, el último enlazado suele ganar (si todo lo demás empata).

```
<link rel="stylesheet" href="base.css">
<link rel="stylesheet" href="theme.css">
```

- `theme.css` se procesa después → puede sobreescribir a `base.css`.

Ojo

- Esto no "rompe" la especificidad: solo decide cuando hay empate.

Ejemplo final integrando los 4 factores

HTML

```
<h1 class="title" id="heroTitle">Bienvenido</h1>
```

CSS (orden en archivo)

```
/* 1) navegador: h1 { font-size: 2em; ... } */

/* 2) autor */
.title { color: blue; }            /* (0,1,0) */
#heroTitle { color: green; }       /* (1,0,0) */
.title { color: orange !important; } /* important */
```

```
/* 3) usuario (por ejemplo, preferencia accesible) podría forzar estilos */
```

Resolución

- Primero compiten las reglas del **autor** frente a las del navegador → ganan las del autor.
- Dentro del autor, `!important` gana a las normales → gana **orange**.
- Si hubiera otra regla `!important` más específica, podría ganar esa.
- Si todo empata, gana la última por **orden**.

Regla de ingeniería

"La cascada se controla, no se pelea"

- Evitar `!important` como herramienta diaria.
- Preferir clases simples (baja especificidad).
- Ordenar CSS por capas (base → componentes → utilidades).

Especificidad (explicación desde cero)

Qué es

- Una "puntuación" que determina qué selector "pesa más".

Regla práctica (nivel inicial)

- ID > Clase > Etiqueta

Ejemplo

```
p { color: blue; }         /* menos específico */
.text { color: green; }    /* más específico */
#x { color: red; }         /* aún más específico */
```

Consejo

- Intenta resolver estilos usando **clases**, no IDs ni `!important`.

Herencia: estilos que "se transmiten"

Qué se hereda a menudo

- `color`, `font-family`, `font-size` (dependiendo del caso)

Qué NO se hereda normalmente

- `margin`, `padding`, `border`, `width`

Ejemplo

```
body { font-family: system-ui; color: #222; }
```

- Los textos dentro heredan tipografía/color sin repetir.

El Box Model (modelo de caja)

El **Box Model** (modelo de caja) es el concepto fundamental de CSS que determina cómo se calcula el tamaño total de los elementos y cómo se posicionan en el diseño de una página web. De acuerdo con las fuentes, este modelo trata a cada elemento HTML como una caja rectangular compuesta por varias capas.

Componentes del Box Model

El modelo se divide en cuatro partes principales que afectan el espacio y el diseño:

- **Contenido (Content):** El área donde aparece el texto o las imágenes.
- **Relleno (Padding):** El espacio transparente que rodea el contenido, dentro del borde.
- **Borde (Border):** Una línea que rodea el relleno y el contenido.
- **Margen (Margin):** El espacio transparente fuera del borde que separa el elemento de otros.

Para facilitar su comprensión, se puede utilizar la analogía de un **paquete físico**: el producto es el contenido, el material de embalaje interno es el *padding*, la caja de cartón es el *border* y el espacio de separación con otros paquetes en un estante es el *margin*.

Cada elemento es una caja

- Contenido → padding → border → margin

Diagrama

```
┌──────────────────────────────┐  ← margin (separación externa)
│  ┌──────────────────────────┐│
│  │  ┌────────────────────┐  │
│  │  │     contenido      │  │
│  │  └────────────────────┘  │
│  │     padding              │
│  └──────────────────────────┘
│     border
└──────────────────────────────┘
```

Ejemplo

```
.box {
  margin: 16px;
```

```
  padding: 12px;
  border: 2px solid #333;
}
```

— `box-sizing`: una corrección esencial

Problema por defecto

- `width` no incluye padding/border → cálculos inesperados.

Solución habitual

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Resultado

- `width` pasa a ser “más predecible”.

El problema del tamaño y el border-box

Un error común entre desarrolladores es no comprender cómo el *padding* y los bordes afectan el tamaño total. Por defecto, CSS suma estos valores al ancho (*width*) y alto (*height*) declarados, lo que puede causar que los elementos no encajen o se desborden de su contenedor.

Para solucionar esto, la práctica profesional recomendada es aplicar globalmente la propiedad **box-sizing: border-box;**. Esto cambia el comportamiento del modelo de caja para que el ancho y el alto declarados incluyan el relleno y el borde, haciendo que el diseño sea mucho más predecible y fácil de gestionar.

Aspectos clave para el desarrollo profesional

- **Colapso de márgenes (Margin Collapsing):** Es un comportamiento específico donde los márgenes verticales de elementos adyacentes se combinan en uno solo en lugar de sumarse.
- **Herramientas de depuración:** Se recomienda utilizar las **DevTools** del navegador (específicamente la pestaña "Computed" o el visor de diseño) para inspeccionar visualmente las dimensiones de cada capa de la caja y detectar errores de espaciado.
- **Accesibilidad:** Es vital no confiar solo en el espaciado visual; se deben usar márgenes y rellenos adecuados para asegurar que los objetivos táctiles (como botones) tengan un tamaño suficiente (al menos 24x24 píxeles) para ser accesibles.

Unidades: px, %, em, rem, vw/vh

px

- Tamaño fijo (no escala con preferencia del usuario).

%

- Relativo al contenedor.

em

- Relativo al tamaño de fuente del elemento.

rem

- Relativo al tamaño de fuente raíz (`html`).

vw/vh

- Relativo al viewport (ancho/alto de ventana).

Regla práctica (inicio)

- Tipografía: **rem**.
- Layout flexible: **%**, **fr**, **auto**, y **rem** para espacios.

Colores en CSS

Formas comunes

```
color: red;
color: #ff0033;
color: rgb(255, 0, 51);
color: hsl(350 100% 50%);
```

Buena práctica

- Elegir 1–2 formatos y ser consistente (HSL es cómodo para ajustar).

Tipografía

Propiedades base

```
body {
  font-family: system-ui, sans-serif;
```

```
  font-size: 16px;
  line-height: 1.5;
}
h1, h2 { line-height: 1.2; }
```

La propiedad **`line-height: 1.5;`** define el espacio entre las líneas de texto

Idea clave

- `line-height` sin unidad suele ser mejor (se escala con el texto).

Display: block, inline, inline-block (crítico)

Los conceptos de **block** e **inline** son fundamentales dentro del módulo de diseño de CSS y se consideran conocimientos esenciales para entender el "flujo normal" de un documento. Aunque las fuentes incluyen estos temas en sus marcos pedagógicos, gran parte de la mecánica específica de estos valores es conocimiento externo que detallaré a continuación para completar la explicación.

1. Display: block

Los elementos de bloque (como `<div>`, `<h1>` o `<p>`) tienen un comportamiento estructural:

- **Nueva línea:** Comienzan siempre en una **nueva línea** del documento.
- **Ancho total:** Por defecto, se expanden para ocupar todo el **ancho disponible** de su contenedor padre.
- **Control de dimensiones:** Permiten definir explícitamente propiedades de `width` (ancho) y `height` (alto), las cuales interactúan directamente con el **Box Model**.
- **Uso profesional:** En la ingeniería de interfaces moderna, los contenedores de bloque se utilizan para establecer contextos de formato avanzados, como **CSS Grid** o **Flexbox**, para gestionar distribuciones complejas.

2. Display: inline

Los elementos en línea (como `<span>` o `<a>`) se comportan de manera más fluida:

- **Misma línea:** No fuerzan un salto de línea y se posicionan **uno al lado del otro** si hay espacio.
- **Ancho del contenido:** Solo ocupan el espacio estrictamente necesario para su contenido (texto o imagen).
- **Limitaciones (Información externa):** A diferencia de los elementos de bloque, los elementos `inline` no aceptan dimensiones de ancho o alto, y sus márgenes verticales no desplazan a otros elementos.

3. Display: inline-block

Este valor actúa como un híbrido entre ambos mundos. Aunque las fuentes lo mencionan como un tema de estudio necesario, las siguientes características técnicas provienen de conocimiento externo:

- **Posicionamiento:** Permite que los elementos se coloquen **uno junto al otro** (comportamiento `inline`).
- **Propiedades de caja:** A diferencia de los elementos puramente `inline`, este valor permite definir `width`, `height`, márgenes y rellenos que son respetados por el diseño (comportamiento `block`).

Relación con el Box Model y la Arquitectura

Independientemente del valor de `display` utilizado, la arquitectura profesional de CSS recomienda:

- **Box Sizing:** Aplicar globalmente `box-sizing: border-box` para asegurar que el relleno (*padding*) y los bordes no alteren el tamaño total calculado de estas "cajas", evitando desbordamientos inesperados.
- **Accesibilidad:** No ocultar elementos necesarios para lectores de pantalla usando `display: none`, ya que esto los elimina por completo del flujo y de la tecnología asistiva; en su lugar, se deben usar técnicas que mantengan la accesibilidad.
- **Rendimiento:** Al animar estos elementos, es preferible utilizar propiedades como `transform` u `opacity` en lugar de alterar el modelo de caja (como cambiar anchos o altos), ya que esto evita ciclos costosos de "layout" y "paint" en el navegador.

block

- Ocupa todo el ancho disponible. Ej: `div`, `p`, `section`.

inline

- No rompe línea, ancho según contenido. Ej: `span`, `a`.

inline-block

- Como inline pero acepta ancho/alto.

Ejemplo

```
a { display: inline-block; padding: .5rem 1rem; }
```

Flujo normal del documento (normal flow)

Qué pasa "si no haces nada"

- Los elementos `block` se apilan verticalmente.
- Los `inline` fluyen en línea y hacen wrap.

Gráfico

```
[header]
[main]
[section]
[footer]
```

Posicionamiento: static, relative, absolute, fixed, sticky

static (por defecto)

- Sigue el flujo normal.

relative

- Se mueve respecto a su posición original (mantiene hueco).

absolute

- Sale del flujo y se posiciona respecto a un ancestro posicionado.

fixed

- Pegado al viewport.

sticky

- Híbrido: se comporta como normal y luego “se pega”.

Ejemplo mínimo

```
.badge {
  position: absolute;
  top: 8px;
  right: 8px;
}
```

Flexbox: alinear en 1 dimensión (fila o columna)

Cuándo usar

- Menús, barras, alineación horizontal/vertical, componentes.

Ejemplo

```
.row {
  display: flex;
  gap: 1rem;
  align-items: center;
  justify-content: space-between;
}
```

Flexbox se define como un sistema de **distribución lineal o unidimensional**. Mientras que CSS Grid se utiliza para estructuras de página complejas en dos dimensiones, Flexbox es la herramienta estratégica para organizar elementos en una sola dirección (ya sea como una fila o una columna).

Para alinear elementos eficazmente en esta única dimensión, las fuentes destacan los siguientes conceptos y propiedades:

Propiedades clave de alineación

El control de la alineación en Flexbox se gestiona principalmente a través de tres propiedades de "Box Alignment":

- `justify-content`: Controla la alineación de los elementos a lo largo del **eje principal** (el eje de la distribución lineal).

- **`align-items`**: Gestiona cómo se alinean los elementos en el **eje transversal** (perpendicular a la dirección principal).
- **`align-self`**: Permite que un elemento individual rompa la alineación grupal definida por el contenedor para posicionarse de forma distinta.

El fin de los "hacks" de margen: La propiedad `gap`

Históricamente, los desarrolladores usaban márgenes para separar elementos dentro de un contenedor flexible. Sin embargo, desde 2021, la propiedad **`gap`** es totalmente compatible con Flexbox.

- Permite definir el **espacio entre los elementos** de forma directa
- Se pueden especificar valores distintos para filas y columnas mediante `row-gap` y `column-gap`.

Conceptos

Flex-direction: Define si la dimensión principal es horizontal (`row`) o vertical (`column`).

Control de proporciones: Las propiedades `flex-grow` (crecimiento), `flex-shrink` (contracción) y `flex-basis` (tamaño inicial) determinan cuánto espacio ocupa cada ítem antes y después de aplicar las reglas de alineación.

Flexbox sigue siendo fundamental para crear **componentes modulares** que se adaptan al espacio disponible, especialmente cuando se combina con **Container Queries**, permitiendo que la alineación cambie no solo según el tamaño de la pantalla, sino según el espacio real del contenedor padre.

Grid: layout en 2 dimensiones (filas + columnas)

Cuándo usar

- Estructuras de página, galerías, dashboards.

Ejemplo

```
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 1rem;
}
```

Gráfico

aside	main

CSS Grid es la base fundamental para el diseño responsivo, permitiendo crear estructuras de página complejas que antes eran imposibles de lograr sin recurrir a una excesiva anidación de elementos o "hacks" de código. A diferencia de Flexbox, que se especializa en distribuciones lineales (unidimensionales), Grid ofrece un **control bidimensional** total sobre filas y columnas.

Conceptos

Control de Estructuras Complejas

CSS Grid permite definir **áreas de plantilla** (`grid-template-areas`) y **líneas de cuadrícula con nombre**, lo que facilita la organización de regiones principales de una página como el encabezado, las barras laterales y el pie de página. Esto reduce significativamente la dependencia de librerías externas para gestionar el diseño estructural.

El Avance del Subgrid

Una de las evoluciones más críticas es el soporte generalizado de **Subgrid**.

- **Funcionamiento:** Permite que los elementos hijos de un contenedor de cuadrícula hereden las definiciones de pista (filas y columnas) del padre.
- **Beneficio profesional:** Esto garantiza que elementos como encabezados o pies de página dentro de diferentes "tarjetas" se alineen perfectamente entre sí, independientemente de la variabilidad de su contenido interno.

Rendimiento y Estabilidad Visual

El uso correcto de Grid impacta directamente en las **Core Web Vitals**:

- **Aspect Ratio:** Combinar Grid con la propiedad `aspect-ratio` permite reservar espacio para imágenes antes de que carguen, previniendo saltos visuales o cambios bruscos de diseño (**CLS - Cumulative Layout Shift**).
- **Funciones Matemáticas:** El uso de `minmax()`, `clamp()`, y unidades como `fr` (fracción) permite que las cuadrículas sean intrínsecamente responsivas, reduciendo la necesidad de múltiples Media Queries.

Responsive desde cero: Media Queries

Idea

- Cambiar estilos según ancho del viewport.

Ejemplo

```
.card { padding: 1rem; }

@media (min-width: 768px) {
  .card { padding: 2rem; }
}
```

Regla práctica

- Empieza móvil (simple) → añade mejoras en pantallas grandes.

Imágenes responsivas en CSS (lo esencial)

Problema típico

- Imágenes que "se salen" del contenedor.

Solución mínima

```
img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

Idea clave

```
max-width controla el ancho máximo permitido de un elemento.
Es un límite superior
   width: fija un ancho (con matices), es un objetivo de tamaño.
   max-width: impone un tope ("hasta aquí puedes crecer, pero no más").
Ejemplo conceptual
.box { max-width: 600px; }
   • La caja puede ser 300px, 450px… pero no 800px.
```

En imágenes: por qué se usa `max-width: 100%`

En un layout responsive, lo típico es evitar que una imagen "reviente" su contenedor.

```
img { max-width: 100%; }
```

Qué significa aquí "100%"

- 100% se calcula respecto al **ancho del contenedor** (el bloque que la contiene).
- Resultado: la imagen **no puede ser más ancha que su contenedor**.

Caso real

- Contenedor: 320px
- Imagen (archivo): 1200px
- Con `max-width: 100%` → la imagen se muestra a **320px** (o menos).

Esto es coherente con la evaluación de "responsive" y "optimización de imágenes" en la práctica.

Espaciado: margin, padding y colapso de márgenes

Lo que necesitas saber ya

- `padding` = espacio interno.
- `margin` = separación externa.

Colapso de márgenes (concepto inicial)

- Márgenes verticales entre bloques pueden "fusionarse".

Ejemplo de hábito seguro

- Poner espaciado en una sola dirección (ej. `margin-bottom`) para evitar sorpresas.

Fondos y bordes

Ejemplo

```
.card {
  background: #fff;
  border: 1px solid #e5e5e5;
  border-radius: 12px;
}
```

Idea clave

- `border-radius` afecta al borde y al background (se ve redondeado).

Selectores de atributos (muy útiles)

Ejemplo

```
input[type="search"] { border-radius: 999px; }
a[target="_blank"]::after { content: " ↗"; }
```

Uso

- Estilos por tipo de input, enlaces externos, data-attributes.

Un selector de atributos permite seleccionar elementos HTML **por el valor de uno de sus atributos**, sin necesidad de añadir una clase extra.

Forma general
`elemento[atributo="valor"] { ... }`

Esto es muy útil en Web Apps porque reduce "clases de estilo" y aprovecha la semántica propia del HTML (forms, enlaces, etc.), algo que encaja con criterios de calidad y claridad de código.

`input[type="search"] { border-radius: 999px; }`
Selecciona **solo** los `<input>` cuyo atributo `type` sea exactamente `"search"`.

Permite aplicar estilos específicos a inputs de búsqueda sin afectar a otros inputs (login, email, etc.).

¿Por qué `border-radius: 999px` hace "píldora"?

`border-radius` redondea las esquinas.
Si usas un valor **muy grande** (como `999px`), el navegador limita el radio al máximo posible para esa caja, produciendo un borde totalmente redondeado (tipo "pill").

```
a[target="_blank"]::after { content: " ↗"; }
```

Selecciona enlaces `<a>` cuyo atributo `target` sea `"_blank"`.

HTML típico

```
<a href="https://..." target="_blank">Abrir en otra pestaña</a>
<a href="/about">Sobre nosotros</a>
```

- El primero abre en nueva pestaña/ventana)

¿Qué es `::after`? (pseudo-elemento)

`::after` crea un pseudo-elemento **virtual** al final del contenido del elemento seleccionado.

- No existe en el HTML.
- Se renderiza como si fuera el "último hijo" del elemento.

```
a[target="_blank"]::after { content: " ↗"; }
```

- Añade un símbolo ↗ después del texto del enlace para indicar:
  "este enlace abre fuera / en nueva pestaña".

Esto es una señal útil para UX y accesibilidad (si se acompaña bien), especialmente en apps con muchos enlaces externos.

Variables CSS (Custom Properties) desde cero

Qué son

- Valores reutilizables definidos con `--nombre`.

Ejemplo

```
:root {
  --space: 1rem;
  --radius: 12px;
}

.card {
  padding: var(--space);
  border-radius: var(--radius);
}
```

Ventaja

- Cambiar diseño sin buscar/reemplazar decenas de reglas.

Transiciones (animación simple y segura)

Ejemplo

```
.button {
  transition: transform 150ms ease;
}
.button:hover {
  transform: translateY(-2px);
}
```

Idea clave

- Transiciones suavizan cambios entre estados (hover/focus).

Mini-ejemplo guiado

Objetivo

- Crear una “card” responsive con título, texto y botón.

HTML

```
<article class="card">
  <h2 class="card__title">CSS desde cero</h2>
  <p class="card__text">Una tarjeta simple y limpia.</p>
  <a class="card__button" href="#">Empezar</a>
</article>
```

CSS (mínimo completo)

```
*, *::before, *::after { box-sizing: border-box; }
img { max-width: 100%; height: auto; display: block; }

:root { --space: 1rem; --radius: 12px; }

.card{
  padding: var(--space);
  border: 1px solid #e5e5e5;
  border-radius: var(--radius);
  background: #fff;
  max-width: 42rem;
}

.card__title{ margin: 0 0 .5rem; font-size: 1.25rem; }
.card__text{ margin: 0 0 1rem; line-height: 1.5; }

.card__button{
  display: inline-block;
  padding: .6rem 1rem;
  border: 1px solid #111;
  border-radius: 999px;
  text-decoration: none;
```

```
}

.card__button:hover{ transform: translateY(-2px); }
.card__button{ transition: transform 150ms ease; }

:focus-visible{ outline: 2px solid; outline-offset: 2px; }
```

¿Qué hace esta regla exactamente?

```
*, *::before, *::after { box-sizing: border-box; }
```

Traducción literal

- `*` selecciona **todos los elementos** del documento.
- `*::before` selecciona **todos los pseudo-elementos `::before`**.
- `*::after` selecciona **todos los pseudo-elementos `::after`**.
- `box-sizing: border-box` cambia **cómo se calcula el ancho/alto** de cada caja.

Por qué es relevante en una Web-App

- Mejora la **predecibilidad del layout** y reduce errores visuales en responsive, algo que se valora en "CSS net i eficient" y "Web Responsive".

El Box Model y el problema por defecto (`content-box`)

Por defecto, el navegador usa:

```
box-sizing: content-box; /* valor por defecto */
```

Con `content-box`:

- `width` **solo** define el ancho del **contenido**.
- Luego se **suman** `padding` y `border`.

Fórmula (content-box)

```
ancho total = width + padding-left + padding-right + border-left + border-right
```

Ejemplo numérico: por qué "se rompe" sin querer

```
.box{
  width: 300px;
  padding: 20px;
  border: 5px solid;
}
```

Con `content-box` (por defecto):

- width (contenido) = 300

- padding total = 20 + 20 = 40
- border total = 5 + 5 = 10

Ancho total real = 300 + 40 + 10 = **350px**

Consecuencia típica

- Si el contenedor mide 300px, la caja **se sale** (overflow) aunque "has puesto width: 300px".

La solución: `border-box` (lo que cambia)

Con:

```
.box{ box-sizing: border-box; }
```

Interpretación

- El `width` pasa a incluir **contenido + padding + border**.

Fórmula (border-box)

```
ancho total = width
contenido = width - padding - border
```

Mismo ejemplo numérico con `border-box`

Si mantenemos:

```
.box{
  width: 300px;
  padding: 20px;
  border: 5px solid;
  box-sizing: border-box;
}
```

Ancho total real = **300px** (no crece)

Ahora el navegador ajusta el contenido:

- contenido = 300 - 40 - 10 = **250px**

Resultado

- No hay sorpresas: "width es el tamaño final visible".

¿Por qué se aplica a `*::before` y `*::after`?

Porque `::before` y `::after` generan cajas reales que también pueden tener:

- `width/height`

- `padding/border`
- `background`

Ejemplo típico

```
.badge::after{
  content:"";
  width: 10px;
  height: 10px;
  border: 2px solid;
}
```

Si no incluyes `*::before, *::after`, esos pseudo-elementos podrían seguir en `content-box` y tener tamaños inesperados frente a tus elementos normales.

Efecto práctico en responsive (por qué "esencial")

En responsive trabajas con:

- `width: 100%`
- grids/flex con columnas
- componentes con padding interno

Sin `border-box`, es fácil que algo al 100% **se pase** por culpa del padding/border.

Con `border-box`:

- `width: 100%` significa "ocupa el contenedor, **incluyendo** padding y border".
- layouts más estables en móvil/tablet/escritorio.

Patrón estándar recomendado (versión completa)

Suele combinarse con un reset mínimo:

```
html { box-sizing: border-box; }
*, *::before, *::after { box-sizing: inherit; }
```

Por qué esta variante

- Hace explícito que todo hereda del `html`, y si algún día cambias el root, se propaga.

Pero la versión que has puesto:

```
*, *::before, *::after { box-sizing: border-box; }
```

también es válida y muy común por simplicidad.

Ingeniería CSS: Arquitectura y Lógica

El problema real: CSS "funciona"… hasta que deja de hacerlo

Observación

- CSS casi siempre "muestra algo", pero eso no implica que sea **estable**, **predecible** o **mantenible**.

Síntomas típicos en proyectos reales

- "Arreglos" con `!important`.
- Selectores cada vez más largos.
- Cambias una cosa y rompes otra ("efecto mariposa").
- Estilos duplicados y contradicciones.

Idea clave

- La ingeniería CSS trata de **controlar la complejidad** y reducir el coste de cambio.

Modelo mental: HTML + CSS = contrato

Contrato (qué promete cada parte)

- HTML expone "ganchos" estables: clases, estados, estructura mínima.
- CSS consume esos ganchos con reglas de baja fragilidad.

La cascada (lo que decide el estilo final)

Cascada: el algoritmo que decide "quién gana"

Cuatro factores (orden conceptual)

1. **Origen** (navegador/autor/usuario)
2. **Importancia** (`!important`)
3. **Especificidad**
4. **Orden de aparición** (lo último suele ganar si empata)

Orden de aparición: el caso más frecuente

Ejemplo

```
.title { color: blue; }
.title { color: green; }  /* esta gana */
```

Idea clave

- Si dos reglas empatan en especificidad e importancia, **la última** gana.

`!important`: qué hace y por qué es peligroso

Qué hace

Regla de oro (nivel ingeniería)

- Si necesitas `!important` con frecuencia, falta **arquitectura** (capas, estados, scopes).

Especificidad: la "puntuación" del selector

Modelo simple (para empezar)

- `#id` gana a `.clase`
- `.clase` gana a `etiqueta`

Modelo técnico (tripleta a,b,c)

- **a** = número de IDs
- **b** = número de clases / atributos / pseudo-clases
- **c** = número de etiquetas / pseudo-elementos

Ejemplos

```
p { color: blue; }
.text { color: green; }
#x { color: red; }
```

HTML

```
<p id="x" class="text">Hola</p>
```

Respuesta razonada

- Gana `#x` porque (1,0,0) > (0,1,0) > (0,0,1)

Selectores "agrupadores" y su efecto: `:is()`

Idea (simplificada)

- `:is()` adopta la especificidad del selector más específico dentro del grupo.

Ejemplo

```
:is(.card, #hero) p { ... } /* aquí pesa como #hero */
```

Consejo

- `:is()` es potente para reducir repetición, pero puede subir especificidad sin darte cuenta.

Patrón base: BEM (Block–Element–Modifier)

Por qué BEM

- Reduce fragilidad, baja especificidad y facilita refactor.

Ejemplo

```
.card { ... }              /* Block */
.card__title { ... }       /* Element */
.card--featured { ... }    /* Modifier */
```

HTML

```
<article class="card card--featured">
  <h2 class="card__title">Título</h2>
</article>
```

Regla de oro en BEM: selectores planos

Evitar

```
.card .card__title { ... } /* encadenar aumenta fragilidad */
```

Preferir

```
.card__title { ... }       /* directo y estable */
```

Resultado

- Menos dependencia del árbol DOM.

Arquitectura de estilos: capas, carpetas y tokens

Capas: base → layout → components → utilities

Idea

- No todo CSS es del mismo tipo. **Capas**

- **Base**: resets, tipografía global, defaults.
- **Layout**: estructuras de página (grid general, header).
- **Components**: piezas reutilizables (card, button).
- **Utilities**: helpers puntuales (ocultar, espaciar).

Gráfico

```
Base (bajo) → Layout → Components → Utilities (alto)
```

`@layer`: gobernar la cascada por capas (concepto previo)

Por qué importa

- Evita que una librería o un “fix” destruya tu sistema.

Ejemplo

```
@layer base, layout, components, utilities;

@layer base { body { font-family: system-ui; } }
@layer components { .card { ... } }
@layer utilities { .u-hidden { display:none; } }
```

Organización 7–1 (visión introductoria)

Estructura típica

- `base/`, `layout/`, `components/`, `pages/`, `themes/`, `vendors/`, `abstracts/` + `main`

Idea clave

- Si no sabes dónde va un estilo, probablemente falta una regla de arquitectura.

Tokens: variables CSS como “sistema de diseño”

Qué son

- Valores reutilizables: color, espacio, radio, tipografía.

Ejemplo

```
:root{
  --space-2: .5rem;
  --radius-2: .75rem;
}
.card{
  padding: var(--space-2);
  border-radius: var(--radius-2);
}
```

Ventaja

- Coherencia + cambios globales controlados.

Responsive: del viewport al componente (puente a Container Queries)

Responsive clásico: Media Queries (viewport)

Qué hacen

- Ajustan estilos según tamaño de pantalla.

Ejemplo

```
@media (min-width: 768px){
  .grid { grid-template-columns: 1fr 2fr; }
}
```

Limitación

- Un componente no siempre sabe su tamaño real por el viewport.

El problema moderno: componentes reusables en contenedores distintos

Caso real

- La misma `card` vive en: sidebar estrecha, main ancho, modal, grid.

Conclusión

- El viewport no describe el espacio del componente.

Gráfico

```
Sidebar (estrecho)  → Card compacta
Main (ancho)        → Card expandida
```

Introducción a Container Queries

Idea

- El componente responde a su **contenedor**, no a la ventana.

Código mínimo (solo concepto)

```
.card { container-type: inline-size; }

@container (min-width: 500px){
  .card { /* versión amplia */ }
}
```

Conexión directa

- Esto prepara la sección avanzada de `@container` y unidades `cqi`.

CSS moderno: lógica, scope y composición (entrada suave)

`:has()` como "condición" relacional (introducción)

Qué permite

- Estilar un elemento en función de lo que contiene.

Ejemplo

```
.card:has(img) { /* layout con imagen */ }
fieldset:has(input:invalid) { outline: 2px solid; }
```

Por qué aparece

- Reduce "clases de estado" generadas por JS en muchos casos.

Nesting nativo: legibilidad sin preprocesador (introducción)

Idea

- Anidar reglas con `&` para evitar repetición.

Ejemplo

```
.card{
  &__title{ font-weight: 600; }
  &:hover{ transform: translateY(-2px); }
}
```

Advertencia

- Anidar ≠ aumentar especificidad: sigue siendo clave mantener selectores simples.

`@scope`: limitar el alcance (por qué existe)

Problema

- “Reglas globales” que se cuelan en otros módulos.

Solución conceptual

- Declarar que ciertas reglas solo viven dentro de un área.

`clamp()`: tamaños fluidos sin mil breakpoints

Qué resuelve

- Tipografía/espaciado que crece de forma suave con límites.

Ejemplo

```
h1 { font-size: clamp(1.2rem, 3vw, 2rem); }
```

Layout base: Flex y Grid

Flex vs Grid (regla de decisión)

Flex

- 1 dimensión (fila o columna). Alinear contenido y distribuir.

Grid

- 2 dimensiones (filas y columnas). Sistemas de página y galerías.

Regla rápida

- “Una fila/columna” → Flex
- “Malla” → Grid

Subgrid (solo idea)

Por qué importa

- Alinear el interior de un componente con la rejilla global.

Accesibilidad como default (para entender la parte WCAG)

`:focus-visible` (la pieza mínima imprescindible)

Problema

- Quitar el outline rompe navegación por teclado.

Solución

```
:focus-visible{
  outline: 2px solid;
  outline-offset: 2px;
}
```

Ejemplo

- Convertir CSS “frágil” en CSS “por componente”.

Versión frágil

```
main section div h2 { margin: 0; }
main section div a { border: 1px solid; }
```

Versión robusta (BEM-like)

```
.card__title{ margin: 0; }
.card__link{ border: 1px solid; }
```

Entrega

- Reescribir HTML con clases + CSS con selectores planos.